A Lock-Free Data Plane and the Signal-to-Seed Bottleneck in Nanopore Adaptive Sampling

Khushal Singh[1] and Ritu Garg[1]

Department of Computer Engineering, National Institute of Technology Kurukshetra, Kurukshetra, Haryana 136119, India
khushal6002@gmail.com, ritu.59@nitkkr.ac.in

Abstract. Oxford Nanopore adaptive sampling ("Read Until") requires a client to decide, within a physical deadline, whether to eject a molecule still transiting the pore. First, the decision budget is not the obstacle: a single-process C++20 data plane with single-copy ingestion, zero-copy propagation and a lock-free, reference-counting arena protocol reaches a median per-chunk decision latency of 124.8 µs [115.6–132.3] against a derived service bound of 1.56 ms, with no drops at genome scale. A basecalling client's bare orchestration (readfish with a no-operation caller) costs 124.3 µs per read-chunk on the same rig—as much as our *complete* signal-space decision. Read Until is a soft-real-time *loss system*: a mean inside budget is consistent with an arbitrary deadline-miss rate. Second, we locate the real limit. Our pipeline resolves 17.6% [16.6–18.8] of reads at a zero-false-positive threshold where RawHash2 resolves 91.8% [91.2–92.4] on identical input. We proposed a seed-survival law $P(\text{key}) = a^E$ in per-event bucket agreement a and key length E, argued it was a lower bound, and then *refuted our own bound*: once event retention is data-dependent the form is $P(\text{key}) \approx (1-g)^E a^E$, where g is the correspondence-failure rate. Under one estimator, RawHash2 achieves $a = 0.911$, $g = 0.092$ and 467 anchors per read at $E = 8$, against our 0.783, 0.196 and 14.9 at $E = 12$. We trace this to three causes, each of which we measure: the two encoders retain events asymmetrically, the quantiser buys too little agreement, and the chainer loses recoverable anchors.

Keywords: nanopore sequencing · adaptive sampling · soft real-time systems · lock-free data structures · raw signal mapping · quantisation

1 Introduction

Nanopore sequencing reads a molecule as it passes through a pore, recording the resulting ionic current in real time. Because the signal arrives while the molecule is still in the pore, the instrument exposes an interface, Read Until, through which a client can request that an unwanted molecule be ejected and the channel freed for another. The benefit is a higher on-target yield with no change to library preparation.

Read Until imposes a deadline, and that deadline is physical rather than conventional. Signal is delivered in fixed-rate chunks, each describing bases that have *already* passed through the pore. A decision that arrives after the molecule has left is not merely late; it is worthless. Let C be the number of active channels, T_c the interval between chunks on one channel, and W the number of chunks after which a decision loses its value. The mean service time available per decision is then

$$S < \frac{W \cdot T_c}{C}, \tag{1}$$

which at $W = 2$, $C = 512$ and $T_c = 0.4$ s gives 1.5625 ms, hereafter 1.56 ms.

The deadline has two consequences for how the problem should be framed, and the paper is organised around them.

Soft real time. A decision does not lose value gradually; its usefulness falls to zero at the deadline. The quantity of interest is therefore the *distribution* of decision latency, not its mean.

A loss system, not a delay system. Under saturation, a client must either queue work or discard it. Queueing turns decisions that would have been useful into decisions that arrive too late, so dropping work is the correct policy, not merely a pragmatic one. The consequence for evaluation is sharp and, we argue, under-appreciated: **a mean comfortably inside the budget is consistent with an arbitrary deadline-miss rate.** A system reporting "mean 124 µs against a 1.56 ms budget" has said nothing about whether it meets its deadline.

Existing Read Until work does report latency, in two forms: signal consumed before a decision (mean bases or chunks per read), and mean processing time per read. What we did not find in the systems we surveyed is a deadline-miss rate against a stated service bound, a latency distribution with tail percentiles, or the fraction of decisions arriving too late to act on. That gap—reporting a distribution against a derived budget, with the instrument and environment characterised well enough for the tail to carry meaning—motivates the first half of this paper.

Contributions.

1. A lock-free soft-real-time data plane for signal-space adaptive sampling, with a reference-counting arena protocol that makes reset-under-dispatch unrepresentable by construction rather than by a timing assumption (Section 3), evaluated as a distribution against a derived budget over nine runs (Section 4).
2. A negative result, explained rather than excused: our pipeline is 5.2× less accurate than a published signal-space mapper on identical input, and we locate the cause before seeding (Section 5).
3. A seed-survival law, including the refutation of our own earlier formulation of it, and an estimator-symmetric measurement of its two parameters for both systems (Section 6).
4. Three measured root causes and the design prescriptions that follow (Section 7).

What this paper does not claim. We do not claim competitive mapping accuracy; we report 17.6% against 91.8% and explain it. We do not claim a novel matching method; quantised-event hashing is established [5, 6]. Nor do we claim that our tail latency can be trusted. All measurements were taken under a hypervisor, and Section 9 sets out what can and cannot be concluded from them. All signal is simulated.

2 Background and related work

The interface. MinKNOW [1] provides Read Until through a bidirectional streaming RPC. The client receives raw signal chunks from each channel and sends its decisions back over the same connection. Since version 6.0, the API identifies reads only by a string, with no numeric read index, which restricts the data structures a client can use.

Basecalling clients. readfish [2] and similar clients basecall each chunk and map the resulting sequence. The approach is accurate and builds on mature tools, but it places both a neural network and a sequence aligner [14] inside the decision loop.

Signal-space mappers skip basecalling. They use a pore model to convert the reference into the signal it is expected to produce, and then match quantised query events against it directly. UNCALLED [3] builds an FM-index over events. Sigmap [4] and RawHash [5] seed and chain over quantised event k-mers. RawHash2 [6] extends this with an adaptive quantiser, and RawAlign [7] adds alignment on top of seeding. Sigmoni [8] classifies reads with an r-index over a small picoamp alphabet. The recent benchmarking framework RawBench [9] divides raw signal analysis into reference encoding, segmentation and matching, and evaluates three segmentation methods: t-test changepoint detection, basecaller move tables and a learned segmenter. We use the same division, but we split matching further into quantisation, seeding and chaining, because our argument needs that level of detail.

The pipeline is ordered, and earlier stages bound later ones. The stages are applied in a fixed order: reference encoding → segmentation → quantisation → seeding → chaining. Each stage sets an upper limit on what the later stages can achieve. For example, information lost during quantisation cannot be recovered by any chaining strategy. Sections 5–7 follow this order.

Streaming decision structures. Misra–Gries [12] and Space-Saving [13] identify frequent items approximately, using a fixed amount of memory. We use a set-associative variant. Its update cost is $O(\text{ways})$ rather than $O(\text{counters})$, which is significant on a latency-critical decision path.

Simulation. Squigulator [10] generates signal with an adjustable spread of dwell times; its default coefficient of variation (CV) is about 0.44. Icarust [11] serves a live Read Until API, but it produces the same number of samples for every k-mer, which corresponds exactly to CV = 0. We modified Icarust to draw dwell times from a lognormal distribution.

3 The data plane

3.1 Design constraints

The design follows from four constraints.

1. *No allocation on the decision path.* The time an allocation takes has no upper bound.
2. *No locks.* A blocked worker still holds a deadline, and it cannot meet it.
3. *State bounded by channel count.* State is $O(\text{channels})$, with per-channel constants fixed at compile time, because read length is set by library preparation and cannot be known in advance.
4. *Drop rather than queue.* Under backpressure the system discards work instead of queueing it, for the loss-system reason given in Section 1.

3.2 Single-copy ingestion, zero-copy propagation

The signal is copied only once, at the transport boundary, where the protobuf runtime writes it into arena-backed storage. After that point, no stage copies the signal again. Workers receive a 56-byte borrowed descriptor holding an absolute sample index, borrowed pointers into arena-owned read-identifier and payload storage, the owning arena slot, a receive timestamp, and a narrow tail of lengths and type codes. Declared in natural reading order the structure is 72 bytes; field-ordered for size it is 56, and a compile-time assertion that it does not straddle a cacheline is what caught the difference.

Two narrowings are guarded at ingest. Chunk length drops from 64 to 32 bits because a chunk is sub-second and 2^{32} samples is about twelve days at 4 kHz. The read-identifier length drops to 16 bits because identifiers are 36-character UUIDs. The absolute sample index is deliberately *not* narrowed: a 72-hour run at 4 kHz already reaches $\approx 10^9$, uncomfortably close to 2^{32}.

3.3 The arena reference-counting protocol

The hazard is reset-under-dispatch: an arena slot recycled while a descriptor referring to it is still in a ring. Because the memory remains mapped and arena-owned, the failure is silent payload corruption rather than a crash.

The protocol removes it structurally (Figure 1). `acquire()` returns a slot at refcount 1, and that initial reference is held by the **reader**, not by a consumer. The reader pushes descriptors to n shard rings, raising the count to $1 + n$, and only then releases its own. The count can therefore reach zero only after every worker reference has been taken and dropped, and the slot cannot be reset while any dispatch is outstanding. No bound on how long a worker takes is required.

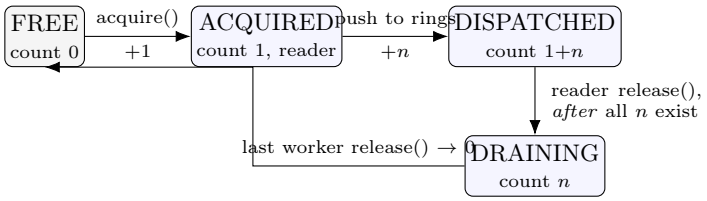


Fig. 1. Arena slot lifetime. The reader holding the initial reference is what makes reset-under-dispatch unrepresentable: the count cannot reach zero while a descriptor is still in a ring, with no assumption about worker duration.

3.4 Lock-free handoff and sharding

Handoff uses single-producer single-consumer rings with free-running 64-bit counters and power-of-two capacity, so the index is a mask rather than a modulus and the full capacity is usable rather than capacity -1. Producer and consumer each cache the other's index, removing cross-core loads from the unsaturated steady state; head, tail and both cached copies are separately cacheline-aligned. Shard rings hold 2048 descriptors; action rings hold 1024. At four shards the rings total under 0.75 MB, negligible beside the index.

Chunks are sharded by `channel & mask`, so all per-channel state is reached by exactly one thread **by construction**. This is what makes the lock-free claim cheap: there is no shared mutable per-channel state to protect. Backpressure is drop-never-block, with the invariant $\mathit{offered} = \mathit{pushed} + \mathit{dropped}$ maintained by counters.

3.5 Bounded per-channel state

The decision structure dominates per-channel state. The set-associative vote table is $64 \times 4 \times 12$ B = 3,072 B per channel (1.57 MB at 512 channels, 8.22 MB at 2675); its update cost is $O(4)$ rather than the $O(256)$ a classical Misra–Gries table of the same counter count would pay on the decision path, and its expected retention threshold is $m/320$ against $m/257$. The anchor-history chainer is 1024×14 B = 14,336 B per channel. Every figure is a compile-time constant times the channel count; nothing depends on read length or elapsed run time.

3.6 Thread topology

Threads are placed from the kernel's SMT sibling sets rather than from processor-index arithmetic. There is a practical reason for this. Our measurement host numbers SMT siblings next to each other (`0-1 2-3 4-5 6-7`). An earlier rule that gave each role "the next logical processor" therefore put the reader and the writer on *the same physical core* while two whole cores were left idle. This numbering is not the same across machines. With two shards the data

Table 1. Regime R1, 37 kb reference, four repetitions per cell. Median across runs, µs.

configuration	p50	p90	p99	p99.9
pinned, 2 shards	16.3	24.7	49.1	158.0
unpinned, 2 shards	15.8	25.1	50.5	165.7
pinned, 4 shards	18.7	31.1	70.4	312.7
unpinned, 4 shards	18.4	31.6	83.8	386.7

plane runs four threads (reader, writer and two workers), which fits on four physical cores. With four shards it runs six, which does not.

4 Evaluation of the data plane

4.1 Method and instrumentation

Latency is the interval from the transport read returning to the decision being reached, recorded with HdrHistogram [15] at three significant figures, bounding relative error at 0.1%. The recorder reports its own backend on every line, so a degraded estimate cannot be mistaken for a measurement.

We adopted this instrumentation to correct an earlier error. An earlier power-of-two histogram reported four 99th percentiles of 111.93, 112.12, 223.66 and 447.59 µs across four runs. At the host's calibrated invariant TSC frequency of 2418.7 MHz the only values representable in that range are 54.19, 108.38, 216.76 and 433.53 µs. Every reported value was a bucket boundary; two runs "agreeing" at 112 µs agreed only that the true value lay in [54.19, 108.38]. The general form is worth stating: *any* instrument whose quantisation is coarse relative to the between-run variance will report stable values and thereby invite an inference of stability.

The harness records per run the virtualisation status, SMT topology, TSC flags, clocksource, kernel command line, governor, turbo state, free storage and load average, and refuses to label a run publishable unless virtualisation is absent, cores are isolated and the governor is set to performance. Every run reported here carries the development-only label.

Host. Intel Core i5-1135G7, 4 physical cores / 8 logical, under WSL2. **Regimes.** R1: a 37 kb reference under constant dwell. R3: a 64 Mb human chr20 reference under variable dwell at the frozen configuration—4 bits × 12 events, minimizer [16] window 1, detected-event segmentation, two shards, pinned.

4.2 Placement and thread count

Thread count matters more than placement. At every percentile, both two-shard configurations are faster than both four-shard configurations (Table 1). Six data-plane threads simply do not fit on four physical cores.

Table 2. Regime R3, nine runs. Median across runs, with observed range, µs.

percentile	median [range]
p50	124.8 [115.6–132.3]
p90	192.8 [183.0–306.0]
p99	370.2 [346.8–1969.3]
p99.9	1224.5 [1044.9–3284.0]
mean	138.2 [131.2–206.0]

Pinning does not lower the median, but it does make the tail reproducible. Across four repetitions, the 99th percentile with pinning was 46.99, 48.32, 49.97 and 50.88 µs, a spread of 8%. Without pinning it was 44.86, 45.52, 55.45 and 104.45 µs, a spread of 133%. What we can fairly say about processor affinity, then, concerns variance and not location.

4.3 Genome scale: regime R3

We made nine runs of 40 s each. In every log, the line the daemon prints at startup confirms the configuration: 63,981,237 reference bases, a geometry of $4 \times 12 = 48$-bit keys, window 1, 63,981,218 minimizers, 32,888,127 distinct keys, and a 512.0 MiB table plus 204.3 MiB of positions.

Peak resident set is 3.70 GiB in every run—the maximum including the index-build transient; the steady-state index structures are 716 MiB. Offered chunk arrival is 604–645 per second, set by the simulator rather than the daemon. **No chunks were dropped and no deadline misses were recorded in any of the nine runs.**

At the median this is a utilisation of **0.080** against the 1.56 ms bound. A 10-chunk read completes its decisions in **1.25 ms**—a derivation, ten chunks times the measured median per-chunk latency, not an independent measurement.

The tail is where the loss-system framing earns its place. Eight of the nine runs sit strictly inside the budget through the 99.9th percentile, at 1.04–1.40 ms. The ninth does not: one run reports p99 = 1969.3 µs and p99.9 = 3284.0 µs, and is also the worst of the nine at p90 and at the mean (Figure 2).

We do not read this as a deadline miss. Across the nine runs the observed *maximum* spans 2.0 ms to 59.4 ms—two orders of magnitude in a quantity the software should not vary in—and, tellingly, the run with the largest maximum is **not** the run with the worst p99; its p99 is 630 µs. Maximum and p99 are not correlated here, which is the signature of externally injected stalls rather than of slow decisions. On a hypervisor without core isolation, this measurement cannot establish that the system meets its deadline, and cannot establish that it misses it. Stating that honestly is part of the contribution.

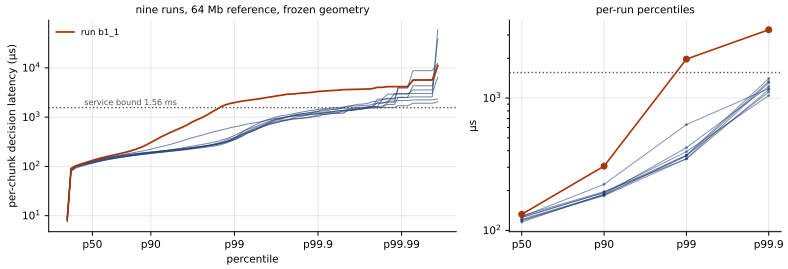


Fig. 2. Regime R3 across nine runs against the 1.56 ms service bound. Eight runs lie inside the budget through p99.9; one crosses at p99 and p99.9.

Table 3. Cost comparison on the same host. readfish figures are its own progress line divided out; processor and memory figures are from `/usr/bin/time -v`.

system	mode	per chunk	CPU	peak RSS
readfish	no-op caller	124.3 µs	15.6%	86.0 MiB
readfish	caller + mapper	66.2 µs	25.3%	663.4 MiB
this work	frozen, 64 Mb	124.8 µs	181.5–202.3%	3786 MiB

4.4 Comparison with existing clients

The architectural point. readfish [2] with a no-operation caller performs no basecalling and no mapping; 124.3 µs is its orchestration alone, a lower bound on its cost. Our *complete* signal-space decision over a 64 Mb index costs 124.8 µs. The orchestration layer of a basecalling client is already as expensive as an entire signal-space decision. This does not establish superiority—a production readfish does work this configuration omits—but it does establish that the decision budget is not what stands between signal-space matching and deployment.

The cost is processor time: 181.5–202.3% of one core against readfish's 15.6%, because workers spin before yielding. We report it alongside.

Against a signal-space mapper. RawHash2 [6], single-threaded on identical reads with the same pore model, requires 10.64 ms of processor time per read including index load, or 10.18 ms excluding the load a long-running daemon pays once. Our comparable figure is the derived 1.25 ms. The ratio is 8.2×, and must be read with Section 5: a latency ratio between a system that resolves reads and one that largely does not is a bound on data-plane cost, not a performance result.

5 The accuracy gap

5.1 Head-to-head

Both systems were run on the same 20 Mb chr20 reference, the same Squigulator [10]-derived R10.4 9-mer pore model on both sides, the same SLOW5 files, three seeds, 500 reads per cell, at Squigulator's default dwell dispersion.

Table 4. Accuracy, three seeds, 500 reads per cell.

system	s42	s43	s44	mean [range]
this work	18.8	17.4	16.6	**17.6** [16.6–18.8]
RawHash2	92.4	91.8	91.2	**91.8** [91.2–92.4]

Table 5. Decision rules at the same operating point, three seeds.

decision rule	s42	s43	s44	mean [range]
votes (256 counters)	14.2	15.0	13.0	14.1 [13.0–15.0]
greedy, $\beta = 0$ (ctrl)	9.6	10.8	7.0	9.1 [7.0–10.8]
greedy, $\beta = 12$	17.4	16.6	15.6	16.5 [15.6–17.4]
anchor-history, $L = 48$	18.8	17.4	16.6	17.6 [16.6–18.8]
anchor-history, $L = 128$	19.6	19.4	18.0	**19.0** [18.0–19.6]
$L = 48$, occ. filter	17.8	15.8	15.4	16.3 [15.4–17.8]

Our figures are true-positive rates at the lowest threshold giving zero false positives on a composition-matched shuffled control. RawHash2's are correctly-mapped fractions at its own default operating point, which is **not** zero-false-positive: 8.4–10.6% of control reads are mapped. The comparison is of magnitude, not of level, and we state it as such.

A factor of 5.2 (Table 4). The rest of this paper accounts for it.

5.2 Replication is not optional at this scale

Table 5 gives six rules in triplicate.

The $\beta = 0$ control is the informative row. With gap tolerance removed, chaining is *worse* than the vote table—9.1% against 14.1%—as expected when it tracks 24 candidates where the table tracks 256. The improvement is therefore attributable to the tolerance, not to the change of data structure.

The best decision rule is +35% **relative over voting**, 19.0% against 14.1%. No rule we tried exceeded that.

Single rules vary by up to 3.8 points across seeds—the control spans 7.0 to 10.8—comparable to the gaps *between* rules, and the ranges of adjacent rules overlap. Any comparison between configurations differing by less than roughly four points requires replication. We learned this the expensive way: an earlier single-run table reported that increasing look-back from 48 to 128 did not help. It does, by 1.4 points of mean and positively in every seed; the single run that supported the old claim was a different seed from the other five rows of the same table.

Even at its best, the decision rule moves accuracy by tens of percent where the gap to be closed is a factor of five. The deficiency is not here.

6 A seed-survival law

6.1 The model, and the refutation of our own bound

Let a be the probability that reference and query assign the same quantiser bucket to corresponding events. A key packs E consecutive events, so a key matches only if all E agree, and expected anchor yield falls as a^E.

We originally argued this was a **lower** bound. Two things make per-event agreements positively correlated. Consecutive events come from overlapping k-mers (two adjacent 9-mers share eight bases). In addition, all events of a read are normalised with one shift-and-scale estimate, so an error in that estimate moves all of the read's z-values together. When agreements are positively correlated they tend to cluster, and a run of E events is then more likely to agree completely than it would be if events were independent.

However, this argument depends on a condition that fails for most of the configurations we study. It assumes that reference and query events correspond one-to-one. A single inserted or deleted event in a key window prevents a match, however well the other events agree. Let g be the probability that an alignment column is an indel rather than a matched pair. The key-match probability is then

$$P(\text{key}) \approx (1-g)^E \cdot a^E, \tag{2}$$

where the first factor is missing from the simple form. With fixed-width segmentation and constant dwell, g is small and a^E is the lower bound argued above; the direct count exceeds it by 1.8× and 5.1× in the two such configurations. With data-dependent segmentation or variable dwell, g rises to 0.09–0.21 and a^E becomes an **over-estimate**, by 1.9× to 14× across the remaining cells (Figure 3).

The qualitative content survives—yield falls geometrically in E, with a base that is a measurable property of the quantiser—but the use of a^E as a one-sided bound is restricted to the exact-correspondence case.

6.2 An estimator-symmetric instrument

An agreement figure computed two different ways for two systems is not a comparison. We therefore emit per-event quantised symbols from both systems in one format and score them with one program, which does not know which system produced its input.

Our symbols come from the production reference-encoding and quantisation paths. RawHash2 [6] ships `-out-quantize`, which dumps query-side symbols; its reference path hardcodes the dump off, so we added a four-line `-out-quantize-ref` exposing the same facility on the reference side, and fixed a record-separator defect that makes the stock multi-read output unparseable.

The scorer aligns the two symbol streams semi-globally. The query is aligned in full, end gaps on the reference are free, and the scores are $+1$ for a match, -1 for a mismatch and -1 for a gap. It reports a under the maximising alignment,

Table 6. Agreement, correspondence failure, and measured key-match probability. 200 reads per cell, identical input, one estimator.

system	CV	a	g	E	P(key)	anchors
this work	0.00	0.797	0.170	12	0.0124	33.8
this work	0.44	0.783	0.196	12	0.0055	14.9
RawHash2	0.00	0.913	0.055	8	0.350	651
RawHash2	0.44	0.911	0.092	8	0.254	467

a under the single best constant offset, the indel fraction, and the fraction of positions that begin a gap-free, all-matching run of E columns, read directly from the alignment path. This last figure is P(key) **measured rather than modelled**, so the independence assumption is no longer needed at all.

Validation. Under the conditions of our earlier per-event diagnostic, the independent instrument returns a constant-offset agreement of 0.556 against the 0.559 previously published: different code, same input, agreeing to 0.3 percentage points.

On the choice of estimator. The constant-offset estimator, which our earlier work used, understates agreement by a factor of four once retention is data-dependent: 0.188 for our frozen configuration where the correspondence-aware figure is 0.783. Results obtained with it for detected-event configurations are withdrawn, including the claim that detection roughly halves agreement. It does not.

6.3 Results

The gap is large and it is before seeding. At CV 0.44, measured key-match probability differs by a factor of **46** (0.254 against 0.0055) and anchor supply by a factor of **31** (467 against 14.9 per read, the two ratios differing because the systems retain different numbers of events per read; Table 6). A 31-fold difference in anchor supply accounts for 91.8% against 17.6% without any appeal to the decision rule.

The key-match difference decomposes, and key length is the smaller lever. Within our pipeline, measured P(key) at $E = 8$ rather than 12 is 0.0274 against 0.0055, a factor of **4.9**. The remaining factor of **9.3** (0.254 against 0.0274 at the same key length) comes from the encoding and segmentation themselves; $4.9 \times 9.3 = 45.8$, which recovers the 46-fold total. Shortening keys is therefore the smaller lever, and, as Section 7 shows, what it gains is paid for in lost specificity.

RawHash2's agreement is dwell-invariant, while ours is not. Across CV 0 to 0.44, RawHash2 goes from 0.913 to 0.911 and we go from 0.797 to 0.783. Over the same range our key-match probability falls by a factor of 2.2, because g rises from 0.170 to 0.196. Correspondence changes; agreement does not.

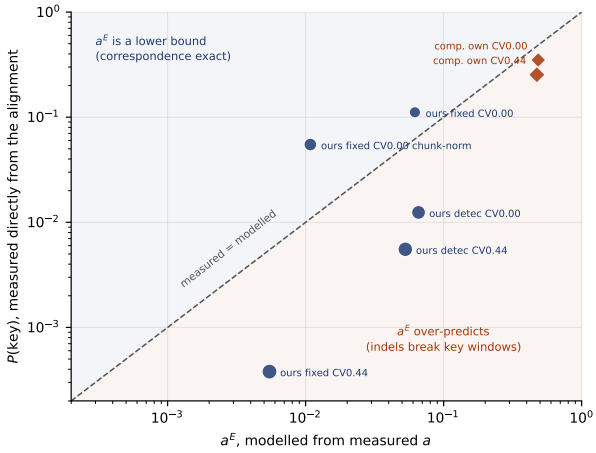


Fig. 3. Measured key-match probability against the a^E prediction, with marker area scaled by the correspondence-failure rate g. a^E is a lower bound only where correspondence is exact.

7 Three root causes

7.1 Asymmetric event retention

RawHash2 [6] merges similar events: an event is dropped if its level differs from the last retained event by less than `-sig-diff` $= 0.35$. The same rule is applied **to both the reference encoding and the query**, and it keeps 73.4% of events on each side. As a result, the reference and the query contain the same number of events for any given span of signal.

Our pipeline does not have this property: its reference encoder outputs one event per k-mer, while its query detector outputs about 0.78. Even at zero dwell dispersion, where there is no timing noise, our indel rate is **0.170**. A rate this high without noise indicates a structural mismatch between the two encoders, not a property of the signal. Correspondence failure enters P(key) with the same exponent as disagreement, so it is just as costly. It is the first problem to fix, and fixing it needs a change to the indexing path, not new research.

7.2 A ≈2.5-bit quantisation sweet spot

With a nominal four bits, RawHash2 could use sixteen buckets. On chr20, however, its symbols fall into only six of them (7.7, 21.1, 9.7, 27.6, 24.0 and 8.8 per cent), plus 0.9 per cent in an outlier bucket, giving a realised entropy of about **2.46 bits per event**. This is by design: the quantiser is non-uniform, with fine resolution in a central band and coarse resolution outside it.

A third system points the same way, which matters because two systems alone could agree by coincidence. Sigmoni [8], which classifies rather than maps and

Table 7. Realised key entropy and occurrence-cap survival, 20 Mb reference.

geometry	nominal	distinct keys	realised	short	cap
3×8	24	92,482	16.5	7.5	2.8%
4×8	32	1,519,844	20.5	11.5	31.2%
3×13	39	5,096,781	22.3	16.7	63.7%
3×14	42	7,985,629	22.9	19.1	78.8%
4×12	48	14,908,778	23.8	24.2	93.5%

shares none of our decision machinery, quantises into an alphabet of **six** symbols by default—2.58 bits—on the stated grounds that it balances robustness to noise against specificity in exact matching. Two independent systems landing within a tenth of a bit of each other, one naming noise robustness as the reason, suggests the operating point is not accidental.

Our uniform quantiser over a clipped z-range spends its alphabet evenly, realises more of it, and agrees less. **The design axis is per-event entropy against per-event agreement, and the two systems sit at opposite ends of it.** A key of 8 events at 2.46 bits carries about 19.7 bits of nominal specificity; one of 12 events at nearly 4 realised bits carries about 48. The system with the *less* specific key is five times more accurate, because specificity it does not need costs it nothing and agreement it does need costs us everything.

Differencing does not help, and the reason is instructive. Encoding the *difference* between successive levels removes the per-read normalisation shift shared by all events. In triplicate, however, it is much worse: 8.3 [3.6–11.8] against 17.6 for the absolute encoding, and worse in every seed. Differencing still produces one symbol per event, so a key still needs E agreements, while each symbol now depends on two noisy levels instead of one. It targets the right quantity with the wrong mechanism. Merging, in contrast, reduces E *and* increases a; since both appear in $(1-g)^E a^E$, it helps twice.

Specificity also depends on scale. A 39-bit key realises 22.3 bits, a shortfall of 16.7; the frozen 48-bit key realises 23.8 bits, a shortfall of 24.2 (Table 7). **The shortfall therefore grows faster than the nominal key width**: doubling the nominal key space from 24 to 48 bits adds only 7.3 realised bits.

Short keys also saturate the occurrence cap, more so on larger references. A 3×8 geometry keeps only 2.8% of minimizer occurrences on the 20 Mb reference (97.2% are truncated), and only 0.7% on the 64 Mb reference (99.3% truncated). Short keys degrade faster than the reference grows, so a geometry chosen on a small reference does not transfer to a large one. This is why raising anchor yield by shortening keys fails: realised entropy collapses before the yield becomes useful.

7.3 Recoverable loss inside the chainer

The correspondence-aware measurement revises our frozen configuration's anchor supply from a modelled 1.85 per read—derived from the simple a^E form

for a *fixed-width* configuration we do not ship—to a measured **14.9** for the configuration we do. The observed median best-chain score is **4**.

So roughly 11 of 15 anchor-eligible positions per read are not reaching the best chain. This loss was invisible while the estimate sat *below* the observation: a decision stage cannot recover more anchors than exist, and 1.85 against 4 left no room for loss. The candidates are the occurrence cap discarding anchors before chaining, the 24-slot greedy chainer thrashing, and a chaining band of 12 against a measured indel rate of 0.196. We have not isolated which.

This qualifies, but does not overturn, the location of the deficiency. Closing a 31-fold gap in anchor supply matters far more than recovering a factor of less than four. But the decision stage is no longer blameless, and because the anchors are known to exist, this is a diagnostic rather than a search.

8 Discussion

Our results lead to four recommendations.

Measure a and g before designing a decision rule. Both can be computed in a single pass over paired reference and query events, without training or search, and together they show whether *any* decision rule can succeed before one is designed. A decision rule reported without these figures cannot be assessed by the reader.

Optimise key-match probability, not agreement alone. Correspondence failure has the same exponent in $P(\text{key})$ as disagreement, and our indel rate is 0.170 even without timing noise. A smaller alphabet can improve both. RawHash2 and Sigmoni make this trade-off; our pipeline does not.

Report latency distributions against a derived budget. When decisions have a deadline, a mean latency within the budget is compatible with any miss rate. We therefore report all nine runs with their full ranges and identify the one run that exceeds the bound.

Run the external baseline first, not last. We evaluated every change to the decision stage against our own earlier results, which varied by plausible amounts and never suggested that the real limit lay elsewhere. A single run of a published tool on the same input would have shown within a day that the gap was a factor of five, not a tuning problem. In an ordered pipeline, internal improvements are measured against the wrong reference point until an external baseline sets the scale.

9 Limitations and threats to validity

All latency figures were measured under a hypervisor. WSL2 can deschedule a vCPU mid-decision, which in the data looks exactly like a slow decision. We therefore treat p50 and p90 as properties of the implementation and p99 and above as upper bounds contaminated by the platform. The direct evidence is the 2.0–59.4 ms spread of observed maxima, uncorrelated with p99. A

deadline-miss rate against the 1.56 ms bound requires bare metal with isolated cores, tickless operation, interrupt affinity moved off the measured cores and a fixed governor. We did not have it, and no claim here depends on the tail.

All signal is simulated, from two generators whose detector calibrations do not transfer to each other by a factor of thirty, and every figure derives from a single reference, human chr20. Agreement is measured at two dwell levels, one seed, 200 reads per cell, where accuracy is swept at six levels in triplicate, so the claim that RawHash2's agreement is dwell-invariant rests on two points. The alignment-based agreement figures are upper bounds, computed under the correspondence that maximises matches; indel fractions of 0.055–0.208 mean the alignments are near-diagonal, so the estimator is not manufacturing matches through heavy gapping, but the bound is one-sided and the asymmetry favours RawHash2. The symmetric-retention explanation is visible in RawHash2's source and consistent with every measurement, but the decisive ablation—applying our detector to our own reference encoding and re-measuring—was not made. RawHash2, Squigulator, readfish and Icarust were built and run here; claims about UNCALLED [3], Sigmap [4], RawAlign [7] and Sigmoni [8] rest on their publications and repositories rather than on execution.

10 Conclusion

We set out to determine whether the adaptive-sampling decision can be made in microseconds. It can: a median of 124.8 µs at genome scale against a 1.56 ms physical budget, with bounded per-channel state, no allocation on the decision path, and no drops at the offered rate—while the orchestration layer alone of a standard basecalling client costs 124.3 µs per read-chunk before any decision is made. The decision budget is not the obstacle to signal-space adaptive sampling.

The question we answer better than the one we asked is why that is insufficient. Quantised-event matching is bounded not by the rate at which decisions can be made but by the fidelity with which signal survives segmentation and quantisation, captured by two measurable quantities whose E-fold product governs everything downstream. At the agreement and correspondence our pipeline achieves, no decision rule placed after quantisation can recover what segmentation and quantisation discard—which is why six attempts to place one moved accuracy by at most 35% relative against a gap of a factor of five.

That is a negative result about this system and a positive one about where to look. We state it without apology: a system that fails for a reason that is measured, modelled, and shown to account for the failure of six attempts to fix it is of more use to the next designer than one that works for reasons its author cannot name.

Reproducibility

All code, data and scripts are in the accompanying repository at commit `eb0ea28`. `scripts/verify_thesis_numbers.py` recomputes every number

from files under `results/`, or, for figures describing code that no longer exists, checks them against the commit message that recorded them; it exits non-zero on any disagreement, and reports 143 claims agreeing and none disagreeing. `scripts/verify_paper_numbers.py` performs the same check against this manuscript. `scripts/generate_thesis_figures.py` regenerates every data figure from `results/` and nothing else, byte-stably. Five claims are not settled by any committed file and are documented as scope boundaries: a bare-metal deadline-miss rate, real instrument data, agreement across the full dwell sweep and across seeds, the Icarust accept rates behind the simulator comparison, and the index resident set at minimizer window 10.

References

1. Oxford Nanopore Technologies, "MinKNOW API." [Online]. Available: `https://github.com/nanoporetech/minknow_api`
2. A. Payne, N. Holmes, T. Clarke, R. Munro, B. J. Debebe, and M. Loose, "Readfish enables targeted nanopore sequencing of gigabase-sized genomes," *Nature Biotechnology*, 2021.
3. S. Kovaka, Y. Fan, B. Ni, W. Timp, and M. C. Schatz, "Targeted nanopore sequencing by real-time mapping of raw electrical signal with UNCALLED," *Nature Biotechnology*, 2021.
4. H. Zhang, H. Li, C. Jain, H. Cheng, K. F. Au, H. Li, and S. Aluru, "Real-time mapping of nanopore raw signals," *Bioinformatics*, vol. 37, no. Suppl. 1, 2021.
5. C. Firtina, N. Mansouri Ghiasi, J. Lindegger, G. Singh, M. B. Cavlak, H. Mao, and O. Mutlu, "RawHash: enabling fast and accurate real-time analysis of raw nanopore signals for large genomes," *Bioinformatics*, vol. 39, no. Suppl. 1, 2023.
6. C. Firtina, M. Soysal, J. Lindegger, and O. Mutlu, "RawHash2: mapping raw nanopore signals using hash-based seeding and adaptive quantization," arXiv:2309.05771, 2023.
7. J. Lindegger, C. Firtina, N. Mansouri Ghiasi, M. Sadrosadati, M. Alser, and O. Mutlu, "RawAlign: accurate, fast, and scalable raw nanopore signal mapping via combining seeding and alignment," arXiv:2310.05037, 2023.
8. V. Shivakumar, O. Ahmed, S. Kovaka, M. Zakeri, and B. Langmead, "Sigmoni: classification of nanopore signal with a compressed pangenome index," *Bioinformatics*, 2024.
9. M. Eris, C. Firtina, O. Mutlu *et al.*, "RawBench: a comprehensive benchmarking framework for raw nanopore signal analysis techniques," in *Proc. ACM Conf. Bioinformatics, Computational Biology, and Health Informatics (ACM-BCB)*, 2025. arXiv:2510.03629.
10. H. Gamaarachchi *et al.*, "Squigulator: simulation of nanopore sequencing signal data," *Genome Research*, 2024.
11. R. Munro, A. Payne, and M. Loose, "Icarust: a real-time simulator for nanopore adaptive sampling." [Online]. Available: `https://github.com/LooseLab/Icarust`
12. J. Misra and D. Gries, "Finding repeated elements," *Science of Computer Programming*, vol. 2, no. 2, 1982.
13. A. Metwally, D. Agrawal, and A. El Abbadi, "Efficient computation of frequent and top-k elements in data streams," in *Proc. Int. Conf. Database Theory (ICDT)*, 2005.

14. H. Li, "Minimap2: pairwise alignment for nucleotide sequences," *Bioinformatics*, vol. 34, no. 18, 2018.
15. G. Tene, "HdrHistogram: a high dynamic range histogram." [Online]. Available: `https://github.com/HdrHistogram/HdrHistogram_c`
16. M. Roberts, W. Hayes, B. R. Hunt, S. M. Mount, and J. A. Yorke, "Reducing storage requirements for biological sequence comparison," *Bioinformatics*, vol. 20, no. 18, 2004.